Chapter 11 — One Task at a Time

The Art of Readable Code — Dustin Boswell & Trevor Foucher

HEM Bellyday
DUK Pagnharith
MON Sreylin

Introduction: What Is the Problem?

- Code that does **many things at once** is hard to understand.
- A single block may initialize objects, clean data, parse input, and apply business logic, all mixed together.
- The reader has to keep *every* task in mind at the same time while reading one line.

Key idea

Code should be organized so that it is doing only one task at a time.

Defragmenting code

- Like defragmenting a hard disk: pieces of the same task are scattered around.
- We move all lines of the same task together into one region.

Theory and Method

Definitions

- **Task** — one distinct thing the code is doing. A task can be *very small*: "make sure the object is valid", "loop through every node", "use a default value".
- **Mixed-task code** — code where several tasks are interleaved line by line.
- **Region** — a group of lines (a function, or just a "paragraph" inside a function) that handles one task.

The method

1. List all of the tasks your code is doing.
2. Separate those tasks as much as you can: into **different functions**, or at least into **different sections** of code.

Example 1: Voting Widget — The Problem

A blog comment can be voted `"Up"` ($+1$), `"Down"` (-1), or no vote `""` (0). When a user changes a vote, `vote_changed(old_vote, new_vote)` updates the score.

```
def vote_changed(old_vote, new_vote):
    score = get_score()
    if new_vote != old_vote:
        if new_vote == "Up":
            score += 2 if old_vote == "Down" else 1
        elif new_vote == "Down":
            score -= 2 if old_vote == "Up" else 1
        elif new_vote == "":
            score += -1 if old_vote == "Up" else 1
    set_score(score)
```

- Short code, but full of intricate details: hard to see off-by-one errors or typos.

Example 1: Voting Widget — One Task at a Time

It looks like one task (update the score), but it is really **two tasks**:

1. **Parse** `old_vote` and `new_vote` into numbers.
2. **Update** `score`.

Task 1 — parse a vote:

```
def vote_value(vote):
    if vote == "Up":
        return +1
    if vote == "Down":
        return -1
    return 0
```

Task 2 — update the score:

```
def vote_changed(old_vote, new_vote):
    score = get_score()
    score -= vote_value(old_vote)  # remove old vote
    score += vote_value(new_vote)  # add new vote
    set_score(score)
```

Result: much less mental effort to convince yourself the code works.

Example 2: Extracting Values from an Object — The Problem

Goal: format a location as `"City, Country"`, e.g. `"Santa Monica, USA"`, from a `location_info` dictionary.

- City: prefer `LocalityName`, then `SubAdministrativeAreaName`, then `AdministrativeAreaName`, else `"Middle-of-Nowhere"`.
- Country: `CountryName`, else `"Planet Earth"`.

```
place = location_info.get("LocalityName")                          # e.g. "Santa Monica"
if not place:
    place = location_info.get("SubAdministrativeAreaName")         # e.g. "Los Angeles"
if not place:
    place = location_info.get("AdministrativeAreaName")            # e.g. "California"
if not place:
    place = "Middle-of-Nowhere"
if location_info.get("CountryName"):
    place += ", " + location_info["CountryName"]                   # e.g. "USA"
else:
    place += ", Planet Earth"
return place
```

- New requirement: for US locations, show the *state* instead of the country (`"Santa Monica, California"`). Adding this to the code above would make it much uglier.

Example 2: Listing the Tasks and Rewriting

The code was doing 4 tasks at once:

1. Extract values from `location_info`.
2. Go through a preference order for "City", default `"Middle-of-Nowhere"`.
3. Get the "Country", default `"Planet Earth"`.
4. Update `place`.

Each task now lives in its own region. After task 1, the long dictionary keys are no longer needed.

```
# Task 1: extract values
town    = location_info.get("LocalityName")
city    = location_info.get("SubAdministrativeAreaName")
state   = location_info.get("AdministrativeAreaName")
country = location_info.get("CountryName")

# Task 2: second half (default, then more specific)
second_half = "Planet Earth"
if country:
    second_half = country
if state and country == "USA":
    second_half = state

# Task 3: first half
first_half = "Middle-of-Nowhere"
if state and country != "USA":
    first_half = state
if city:
    first_half = city
if town:
    first_half = town

# Task 4: combine
return first_half + ", " + second_half
```

Example 2: Another Approach

Once tasks are separated, the code is easier to think about, and better ideas appear.

The `if` chain still hides **two subtasks** woven together:

1. Pick the most preferred value that is available from a list.
2. Use a *different* list depending on whether the country is `"USA"`.

Solution: handle the USA and non-USA cases separately.

```
if country == "USA":
    first_half  = town or city or "Middle-of-Nowhere"
    second_half = state or "USA"
else:
    first_half  = town or city or state or "Middle-of-Nowhere"
    second_half = country or "Planet Earth"
return first_half + ", " + second_half
```

- In Python, `a or b or c` returns the first "truthy" value (not `None`, not empty), so the preference list is now easy to read and update.

Example 3: A Larger Example — `update_counts()`

A web crawler counts statistics after each download. Values hide deep inside a complex `HttpDownload` object and may be missing. The original code jumped between **4 interleaved tasks**: (1) use `"unknown"` as default, (2) detect missing members, (3) extract and convert to string, (4) update `counts`.

```
def update_counts(hd):
    # Task: define default values
    exit_state = http_response = content_type = "unknown"

    # Task: try to extract each value, one by one
    if hd.has_event_log() and hd.event_log().has_exit_state():
        exit_state = exit_state_type_name(hd.event_log().exit_state())
    if hd.has_http_headers() and hd.http_headers().has_response_code():
        http_response = str(hd.http_headers().response_code())
    if hd.has_http_headers() and hd.http_headers().has_content_type():
        content_type = content_type_mime(hd.http_headers().content_type())

    # Task: update counts
    counts["Exit State"][exit_state] += 1
    counts["Http Response"][http_response] += 1
    counts["Content-Type"][content_type] += 1
```

– 4 tasks listed, only 3 regions separated — that is fine. Even partial separation helps a lot.

Further Improvement and Summary

Slicing the problem another way: helper functions

```
def update_counts(hd):
    counts["Exit State"][exit_state(hd)] += 1
    counts["Http Response"][http_response(hd)] += 1
    counts["Content-Type"][content_type(hd)] += 1
```

```
def exit_state(hd):
    if hd.has_event_log() and hd.event_log().has_exit_state():
        return exit_state_type_name(hd.event_log().exit_state(
    return "unknown"
```

- No intermediate variables at all (link to Ch. 9). Both versions are readable: one task at a time.

Summary

- Technique: **do only one task at a time**.
- If code is hard to read, **list all the tasks** it is doing.
- Some tasks become separate functions or classes; others become logical "paragraphs" in one function.
- *How* you separate them matters less than the fact that they *are* separated.
- The hard part is accurately describing all the little things your program is doing.